

Challenges- Forensia- Parte 2

Use CyberChef: <https://toolbox.itsec.tamu.edu/#input=Cg>

Desafío 1

Parece que hay algo oculto en este string:

SUZEe2VzdGVfbXNqX2VyYV9zdXB1cl9zZWNyZXRvfQ==

Rta: Esta en base64, como me di cuenta? por los símbolos, y usa letras de a-z/ A-Z (no necesita usar todas) IFD{este_msj_era_super_secreto}

The screenshot shows the CyberChef web interface. On the left, under the 'Recipe' tab, a 'From Base64' operation is selected. The 'Alphabet' dropdown is set to 'A-Za-z0-9+/' and the 'Remove non-alphabet chars' checkbox is checked. On the right, the 'Input' field contains the string 'SUZEe2VzdGVfbXNqX2VyYV9zdXB1cl9zZWNyZXRvfQ=='. Below the input, the 'Output' field displays the decoded result: 'IFD{este_msj_era_super_secreto}'.

Desafío 2

Otro mensaje oculto:

73 70 68 123 97 115 105 95 113 117 101 95 116 97 109 98 105 101 110 95 108 101 101 115
95 101 115 116 111 125

Rta: Esta cadena se nota que es ASCII, te das cuenta porque los numeros estan separados por espacios lo que dice que cada numero es un caracter individual. Notas que los numeros van del 68 al 125 y el rango ASCII imprimible va de 32 a 126. No hay valores mayores a 126.

En cyberChef From Decimal. IFD{asi_que_tambien_lees_esto}

Recipe

From Decimal

Delimiter

Space

☐ Support signed values

Input

73 70 68 123 97 115 105 95 113 117 101 95 116 97 109 98 105 101 110 95 108 101
101 115 95 101 115 116 111 125

raw 110

2

Tr

Raw Bytes

Output

IFD{asi_que_tambien_lees_esto}

Desafío 3

4946447b63616c656e74616e646f5f6d6f746f7265737d

Rta: esto esta en hexadecimal, notar que es una cadena sin espacios con la nomenclatura de a-f/ A-F y 0-9

IFD{calentando_motores}

From Hex

Delimiter

Auto

4946447b63616c656e74616e646f5f6d6f746f7265737d

raw 46

1

Output

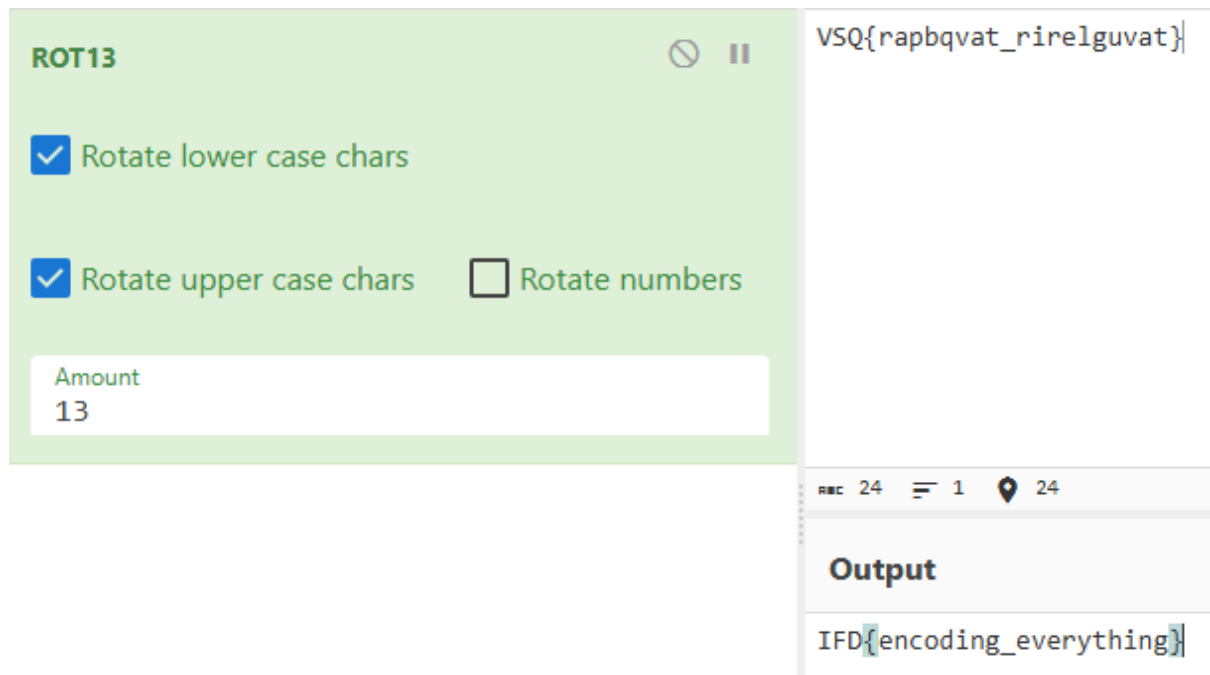
IFD{calentando_motores}

Desafío 4

VSQ{rapbqvav_rirclguvat}

Rta: es algun tipo de ROT porque tiene el formato de letras y simbolos. Los simbolos no cambian.

IFD{encoding_everything}



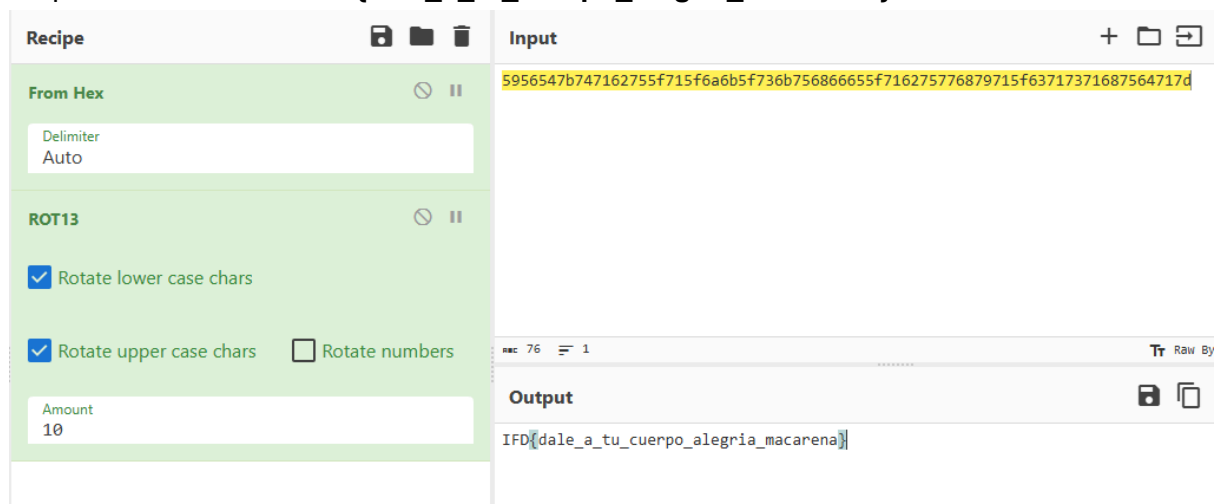
Desafío 5:

Detectamos esto en el tráfico de nuestra red y necesitamos que nos digas que significa:

5956547b747162755f715f6a6b5f736b756866655f716275776879715f63717371687564717d

rta: Es una cadena en hexa que te da como resultado

→YVT{tqbu_q_jk_skuhfe_qbuwhyq_cqsqhudq} y a esto le haces ROT13 con un desplazamiento de 10. **IFD{dale_a_tu_cuerpo_alegria_macarena}**



Desafío 6

Y si te damos un hash?

200ceb26807d6bf99fd6f4f0d1ca54d4

AVISO: la flag de este reto no esta en el formato comun de IFD{}

Desafío 7:

El hash de la password de mi tia:

881486ba0abc356fa032acd1bf7d37ddb6ea3e34fe271f3133ea2ec1dca4fbb6

AVISO: la flag de este reto no esta en el formato común de IFD{} !

Rta: es un hash con 64 caracteres hexadecimales → probablemente es un SHA-256. No se descifra como un cifrado. Lo que hacés es comparar el hash que te dan con los hashes de posibles contraseñas. O sea con el archivo rockyou.txt

Script → challenge7.py

```
import hashlib

# Hash objetivo
hash_objetivo =
"881486ba0abc356fa032acd1bf7d37ddb6ea3e34fe271f3133ea2ec1dca4fbb6"

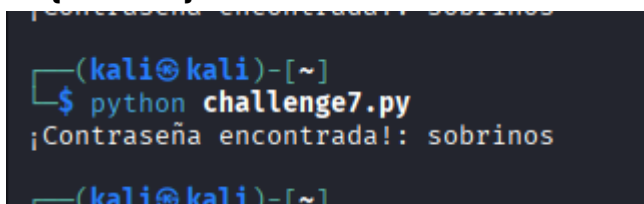
# Ruta al archivo rockyou.txt
archivo = "rockyou.txt"

# Abrir el archivo y recorrer cada línea
with open(archivo, "r", encoding="latin-1") as f:
    for linea in f:
        password = linea.strip()
        hash_generado = hashlib.sha256(password.encode()).hexdigest()
        if hash_generado == hash_objetivo:
            print(f"¡Contraseña encontrada!: {password}")
            break
```

Detalles importantes

- encoding="latin-1": evita errores con caracteres especiales en rockyou.txt.
- strip(): elimina saltos de línea.
- hashlib.sha256(...).hexdigest(): genera el hash SHA-256 en formato hexadecimal.
- break: detiene el script al encontrar la coincidencia.

IFD{sobrinos}



```
(kali㉿kali)-[~]
$ python challenge7.py
¡Contraseña encontrada!: sobrinos
(kali㉿kali)-[~]
```

Desafio 8:

camara.jpg

Rta: Descargue la imagen, puse el comando strings camara.jpg | grep -i fd

El comando strings archivo.jpg extrae todas las cadenas de texto legibles (ASCII) que estan embebidas en el archivo .jpg, incluye comentarios, metadatos, texto oculto, nombres de archivos ..

IFD{exif_botonea_mucho}

```

(kali㉿kali)-[~/Downloads]
$ ls
camara.jpg dino dino.zip ic merged_image.jpg

(kali㉿kali)-[~/Downloads]
$ strings camara.jpg | grep -i ifd
IFD{exif_botonea_mucho}

```

Desafío 9:

Te paso esta imagen para el trabajo que tenemos la semana que viene. Acordate que la clave es re simple.

dino.zip

Probé en steganography online pero no había ningún mensaje oculto ahí. (sacado de la explicacion practica)

Rta: Probé con <http://futureboy.us/stegano/decinput.html> tampoco me dio un mensaje.

(sacado de la explicacion práctica)

Use el comando steghide extract -sf dino.jpg herramienta sugerida en la explicación práctica
clave: resimple. (con el punto final)

IFD{st3g0_s3cr3t}

```

(kali㉿kali)-[~/Downloads/dino]
$ steghide extract -sf dino.jpg
Enter passphrase:
wrote extracted data to "flag.txt".

(kali㉿kali)-[~/Downloads/dino]
$ cat flag.txt
IFD{st3g0_s3cr3t}

(kali㉿kali)-[~/Downloads/dino]
$ █

```

Desafío 10:

Te buscan los de organización de computadoras:

0100100101000110010001000111101101111001001101000101111101100010001101000111
001100110001011000110010000101111101

rta: IFD{y4_b4s1c!}



Desafio 11:

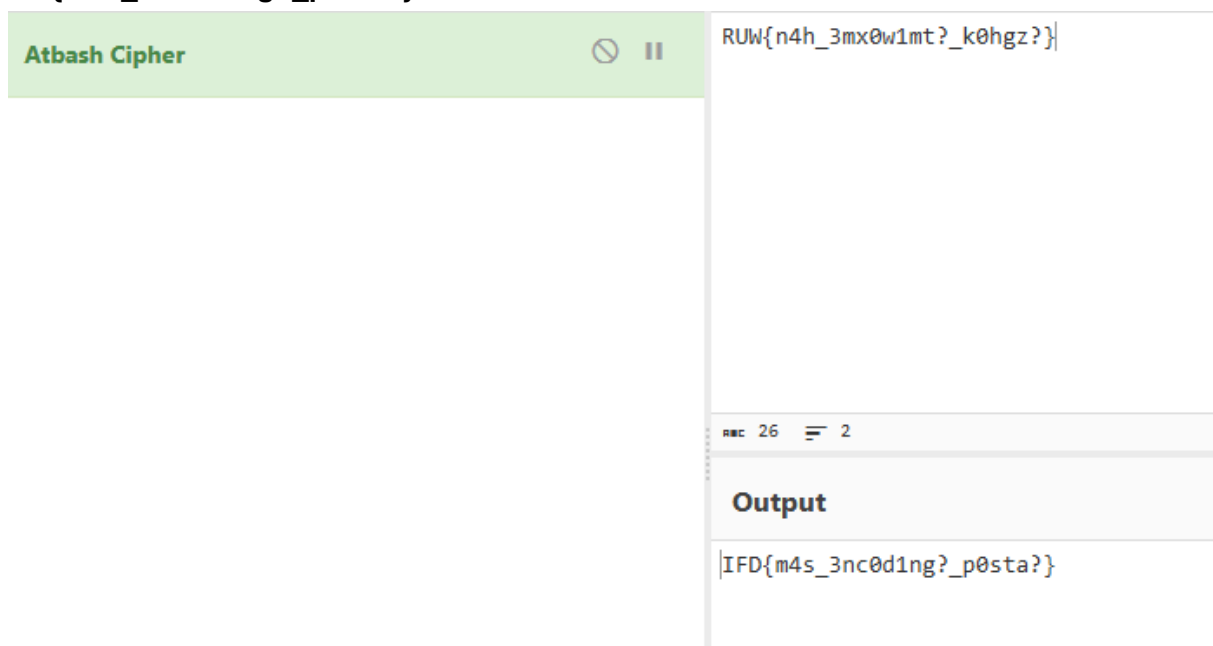
RUW{n4h_3mx0w1mt?_k0hgz?}

rta: Es con cifrado atbash, Es un cifrado por sustitución simétrica que invierte el alfabeto. O sea: la A se convierte en Z, la B en Y la C en X, a Z en A ..

Normal: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Atbash: Z Y X W V U T S R Q P O N M L K J I H G F E D C B A

IFD{m4s_3nc0d1ng?_p0sta?}



Desafío 12:

Hay algo roto en este archivo y no puedo verlo.

File signature / Magic number

Unlock Hint for 0 points

DB

Unlock Hint for 0 points

Hasta donde recuerdo era una imagen.

Unlock Hint for 0 points

La signature no parece ser de lo más estándar para ese archivo.

File //descargar

```

(kali㉿kali)-[~/Downloads]
└─$ unzip file
Archive:  file
  End-of-central-directory signature not found.  Either this file is
  a zipfile, or it constitutes one disk of a multi-part archive.  In
  latter case the central directory and zipfile comment will be found
  in the last disk(s) of this archive.
unzip:  cannot find zipfile directory in one of file or
        file.zip, and cannot find file.ZIP, period.

(kali㉿kali)-[~/Downloads]
└─$ xxd file | head -n 5
00000000:  ffff ffff 0043 0003 0202 0302 0203 0303  .....C.....
00000010:  0304 0303 0405 0805 0504 0405 0a07 0706  .....
00000020:  080c 0a0c 0c0b 0a0b 0b0d 0e12 100d 0e11  .....
00000030:  0e0b 0b10 1610 1113 1415 1515 0c0f 1718  .....
00000040:  1614 1812 1415 14ff db00 4301 0304 0405  .....C.....

(kali㉿kali)-[~/Downloads]
└─$

```

Prueba 1: unzip file. Esto intenta descomprimirlo directamente. Si el archivo es realmente un ZIP con cabecera alterada, puede que lo abra igual o de un error.

El archivo intenta comportarse como un ZIP, pero su cabecera está rota o incompleta.

Los primeros bytes son: FF FF FF FF 04 03 02 02.

Esto es importante porque es parte de la cabecera de un archivo JPEG, del segmento de tabla de cuantización. Pero el inicio correcto de un JPEG debería ser: FF D8 FF DB

Así que lo que hago es reparar la cabecera JPEG manualmente. Hago una copia del archivo original cp file file_reparado.jpg. Remplazo los primeros bytes por: FF D8 FF DB. Guardo el archivo y lo abro: xdg-open file_reparado.jpg //me sale una imagen negra

NO ESTA TERMINADO. no encontré el flag

```

(kali㉿kali)-[~/Downloads]
└─$ xxd file_reparado.jpg | head -n 5
00000000:  ffd8 ffdb 0010 4a46 4946 0001 0203 0303  .....JFIF.....
00000010:  0304 0303 0405 0805 0504 0405 0a07 0706  .....
00000020:  080c 0a0c 0c0b 0a0b 0b0d 0e12 100d 0e11  .....
00000030:  0e0b 0b10 1610 1113 1415 1515 0c0f 1718  .....
00000040:  1614 1812 1415 14ff db00 4301 0304 0405  .....C.....

(kali㉿kali)-[~/Downloads]
└─$ xdg-open file_reparado.jpg

```

Desafío 13:

44fedef642ad4e30bdccbc8a8862baca

Formato: IFD{algoritmo}

5 intentos máximo

Rta: tiene 32 caracteres hexadecimales, o sea es MD5 (128 bits)

probé un script que comparaba el hash MD5 con el hash MD5 de las contraseñas del archivo rockyou.txt pero no dio ninguna conciencia.

IFD{md5}

Desafío 14:

Ojo que este también tiene solo 3 intentos, no te equivoques:

95d63e0d93e88182a520e2b232f1cbe142a86c384cac1669018e9649a2db53877bdf0cc8430982a0cb74d453461214dd74da45ec8d290961ec7bccd8af947f25

Formato: IFD{algoritmo-mensaje_en_texto_plano}

Es con un guión entre ambos y sin espacios

0/3 attempts

Rta: La cadena tiene 128 caracteres hexadecimales. Cada carácter hexadecimal representa 4 bits $\rightarrow 128 \times 4 = 512$ bits. Esto coincide con el tamaño de un hash generado por SHA-512. Use para crackearlo <https://crackstation.net/>

Hash	Type	Result
95d63e0d93e88182a520e2b232f1cbe142a86c384cac1669018e9649a2db53877bdf0cc8430982a0cb74d453461214dd74da45ec8d290961ec7bccd8af947f25	sha512	ctf

IFD{sha512-ctf}

Desafío 15

hash sha1 de la imagen

https://upload.wikimedia.org/wikipedia/commons/b/b8/Diego_Maradona_2017.jpg

Formato IFD{}

5 intentos maximo

Rta: guardas la imagen del link y la pones en cyberChef con SHA1

The screenshot shows the CyberChef web application. On the left, the 'Recipe' panel is set to 'SHA1' with 'Rounds' set to '80'. The 'Input' panel shows a file named 'Diego_Maradona_2017.jpg' with a preview of the image. The 'Output' panel displays the resulting SHA1 hash: 'df7147f23cb711f3db4765a043008e8dab3032f8'.

IFD{df7147f23cb711f3db4765a043008e8dab3032f8}

Desafío 16:

El sólo habla en SHA384 hay q decirle "puede darme la flag por favor?"

Formato IFD{}

Rta: Hice un script tp1_ejer3.py para que pase el texto a SHA384 porque [CyberChef](#) no tenía esa opción de hashing.

IFD{7e100adcec2cb382a266d83d2e9a614b891f12c9ed2221ba50e6300cea56215e41affabfdab481dcbfcb6fe9c5660fe3}

```
(kali㉿kali)-[~/forensia]
```

```
$ ls
```

```
tp1_ejer3.py
```

```
(kali㉿kali)-[~/forensia]
```

```
$ nano --linenumbers tp1_ejer3.py
```

```
(kali㉿kali)-[~/forensia]
```

```
$ python tp1_ejer3.py
```

```
7e100adcec2cb382a266d83d2e9a614b891f12c9ed2221ba50e6300cea56215e41affabfdab4
```

```
(kali㉿kali)-[~/forensia]
```

```
$ cat tp1_ejer3.py
```

```
import hashlib
```

```
mensaje = "puede darme la flag por favor?"
```

```
hash_sha384 = hashlib.sha384(mensaje.encode()).hexdigest()
```

```
print(hash_sha384)
```